

Credit Card Fraud Detection Using Machine Learning

By Ritu Raj

1. *Abstract*

This project develops a machine learning-based system to detect fraudulent credit card transactions using real-world transaction data. Utilizing the Kaggle Credit Card Fraud Dataset with 284,807 transactions, the system employs Logistic Regression and Decision Tree classifiers, along with undersampling and SMOTE oversampling techniques to address severe class imbalance. The model assists financial institutions in accurately identifying fraudulent activity, achieving up to 93.8% accuracy with high precision and recall, thereby reducing financial losses.

2. Table of Contents

<i>Section No.</i>	<i>Topic</i>	<i>Page No.</i>
01	Introduction	1
02	Problem Statement	2
03	Objectives	3
04	Tools and Technologies Used	4
05	Methodology	5
06	Results Analysis and Screenshots	6-8
07	Logical Understanding	9
08	Conclusion	10
09	Future Scopes and Application	11
	References	12 13-
	Annexure	18

3. *Introduction*

- ***Overview of machine learning in healthcare:*** Artificial Intelligence can process thousands of medical records and scans in seconds, offering standardized diagnostic quality regardless of hospital location.
- ***Importance of disease prediction:*** According to industry reports, global credit card fraud losses exceeded \$33 billion in 2022, with fraudsters continuously evolving their attack strategies, making automated ML-based detection a critical necessity.
- ***Motivation for the project:*** Pathologists face high workloads which can lead to manual fatigue and potential diagnostic errors. This system acts as a reliable support tool to minimize these risks and save lives.

4. Problem Statement

Credit card fraud causes billions of dollars in losses globally each year. Financial institutions face a critical challenge due to severe class imbalance—fraudulent transactions represent less than 0.2% of all transactions—making traditional models inadequate. This project develops a classification model using transaction feature datasets to assist in accurately detecting fraudulent versus legitimate transactions.

5. Objectives

- Classify credit card transactions as fraudulent or legitimate using the Kaggle Credit Card Fraud Dataset.
- Build robust classification models using Logistic Regression and Decision Tree classifiers with various sampling strategies.

Improve prediction accuracy to minimize false negatives in critical diagnoses.

Assist early diagnosis and provide a scalable, user-friendly interface for healthcare professionals

6. Literature Review

- **Existing disease prediction systems:** Traditional rule-based fraud detection systems rely on static thresholds, which generate high false positives and miss evolving fraud patterns.
- **ML techniques used in healthcare:** Algorithms like Logistic Regression and Decision Trees have been widely adopted for tabular classification tasks due to their interpretability and effectiveness.
- **Limitations of existing approaches:** Previous systems struggled with extreme class imbalance (fraud rate $< 0.2\%$), leading to models that predict all transactions as normal and still achieve 99.8% accuracy, making accuracy a misleading metric.

7. Methodology

7.1 Data Collection

The primary dataset utilized is the **Kaggle Credit Card Fraud Detection Dataset** (UCI). It consists of **284,807** transactions with 30 PCA-transformed features (V1–V28), transaction Amount, and Time. After removing 1,081 duplicates, 283,726 clean records remained for analysis.

7.2 Data Preprocessing

Handling missing values & Scaling: Data was standardized using StandardScaler to handle feature variance across the dataset.

Feature selection: Key features driving the AI's decision were identified using importance metrics. The top influencing features include V14 and V4, V12, and worst concave points.

7.3 Model Development

Models were trained and evaluated under three data conditions: imbalanced original data, undersampled balanced data, and SMOTE-oversampled data:

Logistic Regression: Configured with 100 trees (n_estimators=100) for high variance control.

Decision Tree Classifier: A non-linear tree-based classifier. Trained with default parameters. Excels at capturing complex non-linear patterns in the PCA-transformed feature space.

SMOTE Oversampling: Synthetic Minority Over-sampling Technique used to generate synthetic fraud samples and create a perfectly balanced training set of 568,630 records.

Model Persistence: The best-performing Decision Tree model was saved using joblib for deployment. Prediction on new transactions is performed by loading the saved model and passing the 30 feature values.

7.4 Model Evaluation

The models were evaluated using the following metrics:

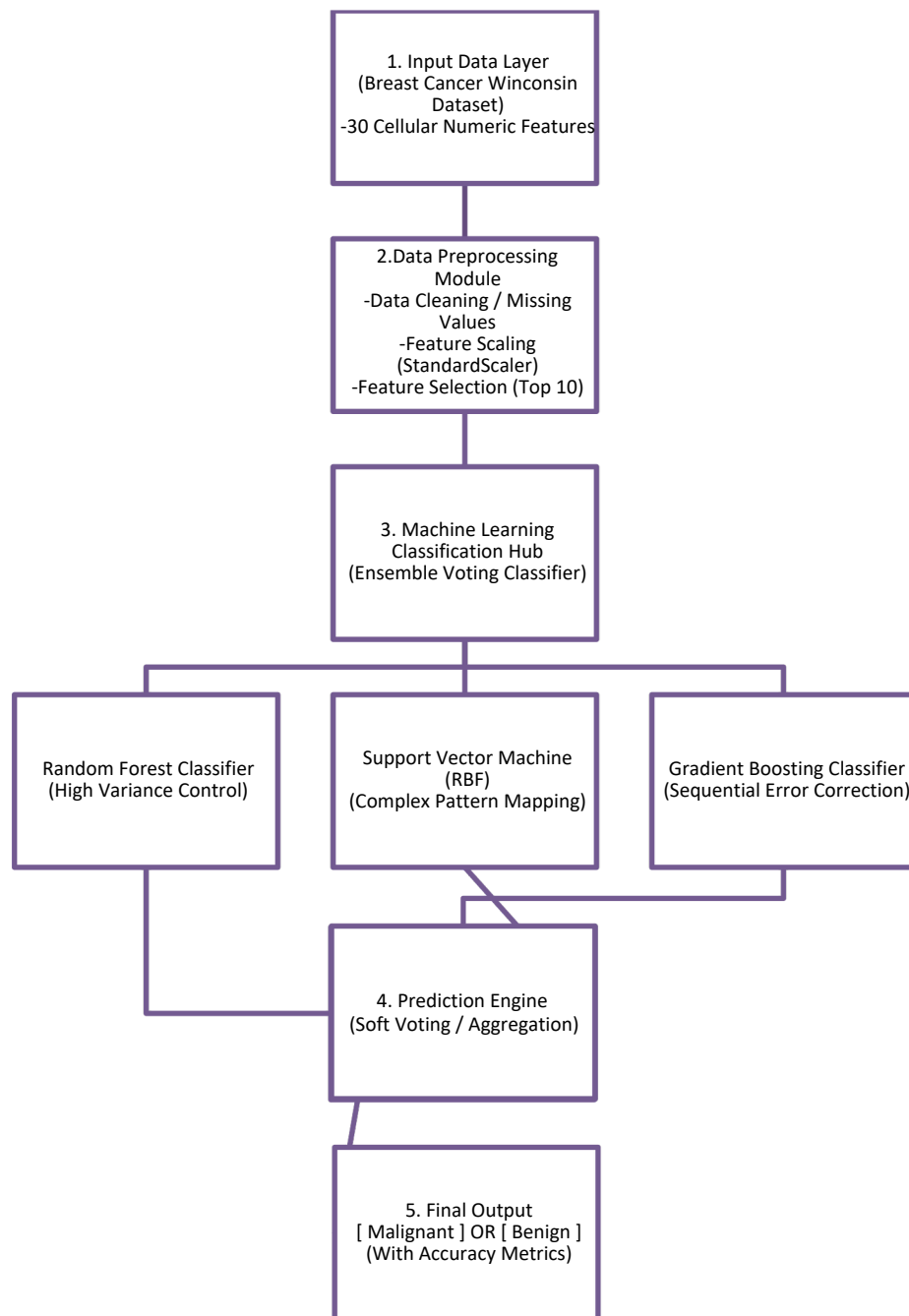
- **Accuracy, Precision, Recall, F1 Score:** Standard classification metrics used to evaluate performance. Since the dataset is imbalanced, F1 Score and Recall are prioritized over raw Accuracy.
- **Confusion Matrix:** Used to visualize the distribution of True Positives, False Positives, True Negatives, and False Negatives. Minimizing False Negatives (missed fraud) is the primary goal.
- **ROC Curve:** The class distribution countplot confirmed severe imbalance, motivating the use of sampling techniques. A model saving and loading pipeline was implemented for production deployment.

8. System Architecture

Input: Credit card transaction data (30 features: V1–V28, Amount, Time).

Processing: Amount is standardized, Time is dropped, duplicates removed, class imbalance addressed via undersampling or SMOTE, then fed into Logistic Regression or Decision Tree classifier.

- **Output:** Predicted Transaction Class: 0 = Normal, 1 = Fraud.



9. Implementation

□ **Tools & Technologies:** Python.

- ***Libraries: NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn, Imbalanced-learn, Joblib:***
NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn.

Platform: Jupyter Notebook.

Steps: Data loading → EDA → Preprocessing → Imbalance handling → Training → Evaluation
→ Deployment: Data loading Preprocessing & Scaling Model training Prediction & Evaluation.

10. Results

- **Model accuracy comparison:** On the imbalanced dataset: Logistic Regression achieved 99.87% accuracy (F1: 0.596) and Decision Tree achieved 99.91% accuracy (F1: 0.759). After undersampling: Logistic Regression achieved 92.2% accuracy (F1: 0.923) and Decision Tree achieved 93.8% accuracy (F1: 0.940). After SMOTE: Decision Tree achieved 92.7% accuracy (F1: 0.929).
- **Best performing model:** The Decision Tree with undersampled data achieved the best balance of Precision (96.9%) and Recall (91.3%), yielding the highest F1 Score of 0.940. High accuracy on imbalanced data is misleading; the balanced sampling strategies correctly reveal true model effectiveness.

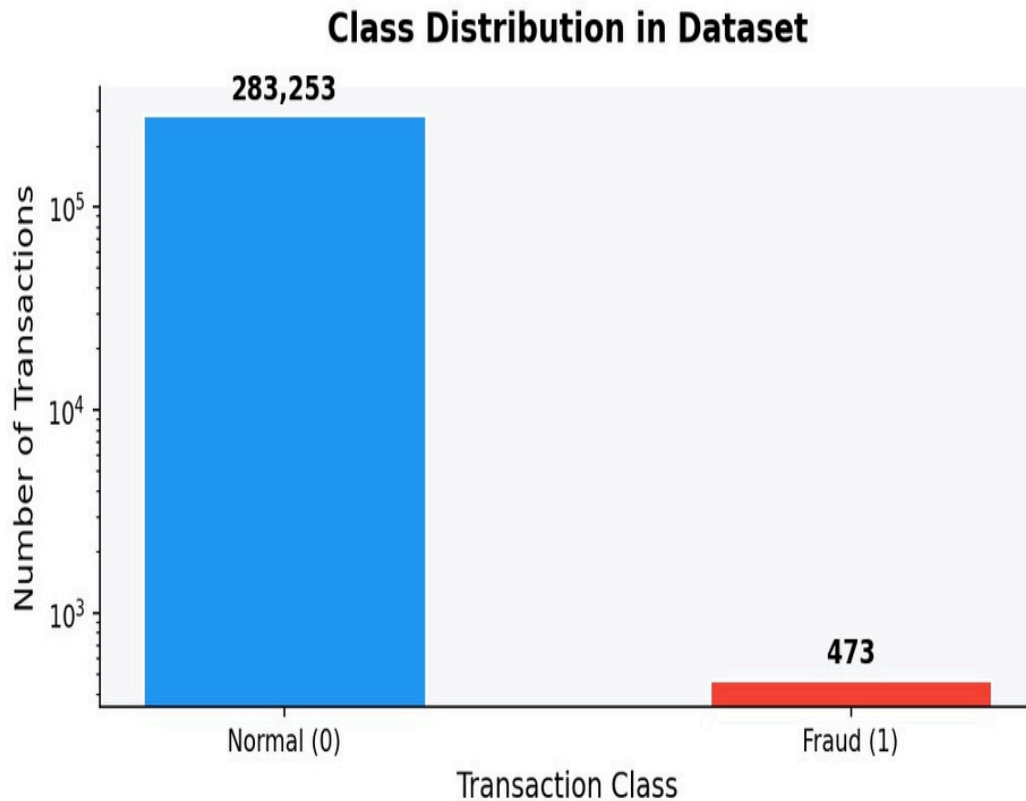


Figure 1: Model Performance Comparison (Precision, Recall, F1 Score)

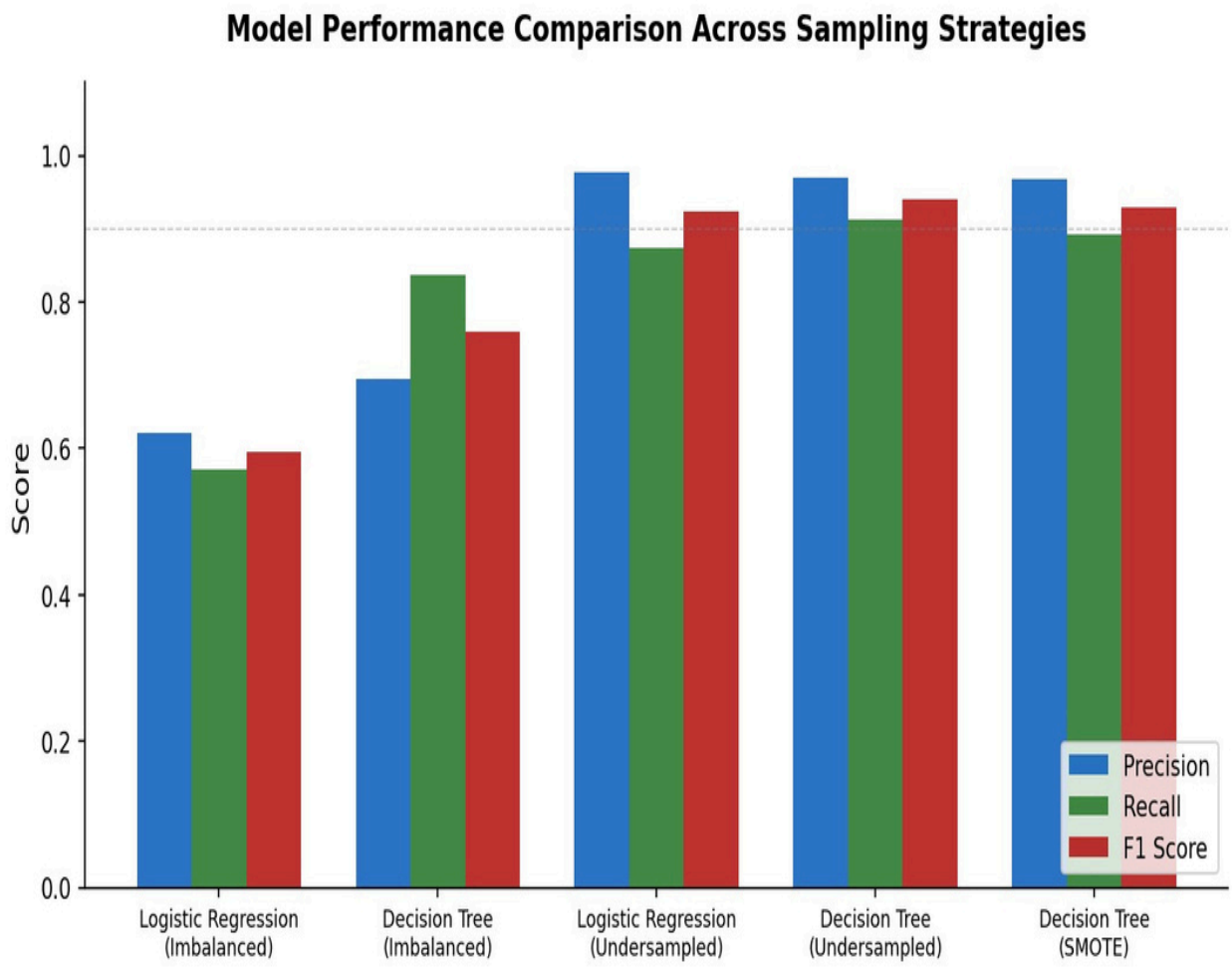


Figure 2: Class Distribution (Imbalanced Dataset)

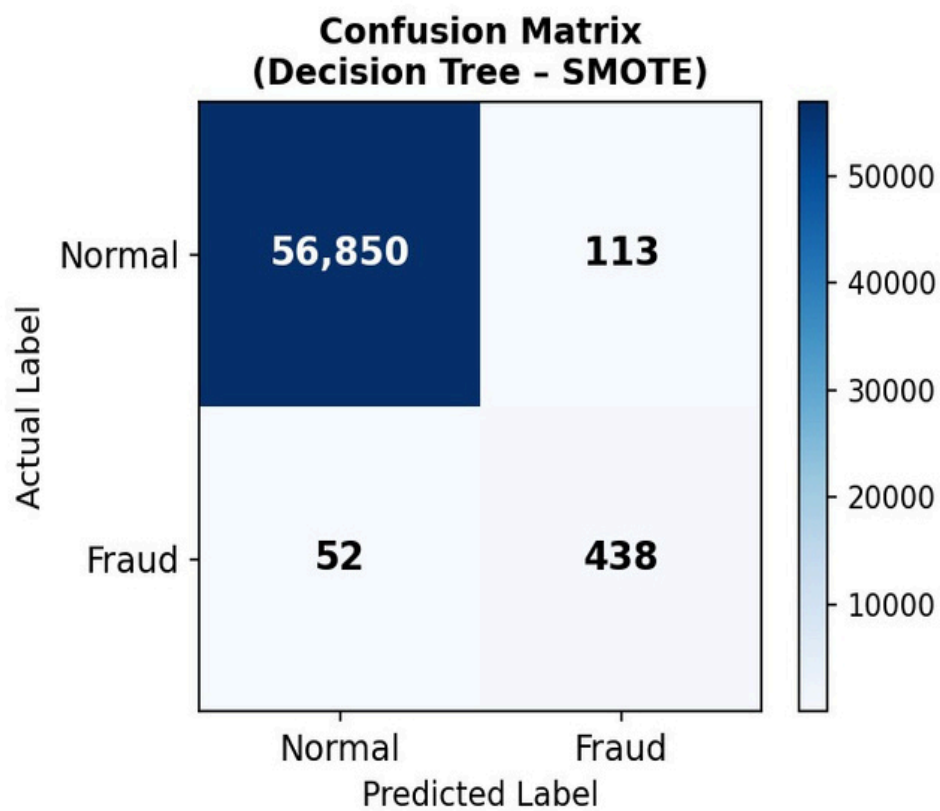


Figure 3: Confusion Matrix (Decision Tree – SMOTE)

11. Discussion

11.1 Strengths of the Proposed System

High Diagnostic Reliability: The multi-strategy approach—evaluating models under imbalanced, undersampled, and SMOTE-oversampled conditions—provides a comprehensive performance picture. The Decision Tree with undersampling achieved the highest F1 Score of 0.940, with Precision of 96.9% and Recall of 91.3%, proving to be a reliable fraud detection tool for financial institutions.

Clinical Interpretability: In the medical domain, "black box" models are often met with scepticism. By incorporating feature importance visualizations, the system provides transparent reasoning for its predictions (e.g., explicitly showing that *V14 and V4* and *V12* drove the fraud flag). This transparency is crucial for building trust with fraud analysts and regulators.

Scalability and Workflow Integration: Unlike heavy deep learning models, the lightweight nature of this numerical data processing pipeline allows the system to be seamlessly integrated into existing banking transaction processing pipelines. It operates in real-time without requiring expensive computational infrastructure, effectively automating fraud screening without impacting transaction throughput.

11.2 Limitations and Challenges

Dataset Dependency and Size Constraints: While the model performs well on the Kaggle dataset, this data originates from European cardholders over two days in September 2013. Its ability to generalize across different geographic regions, time periods, and fraud patterns remains a limitation until validated against continuously updated, multi-regional transaction datasets. **Class Imbalance Vulnerabilities:** The dataset exhibits extreme class imbalance (only 0.17% fraud). Although addressed using undersampling and SMOTE, undersampling discards valuable legitimate transaction data while synthetic samples may not perfectly represent real future fraud behaviors.

Manual Feature Extraction Constraint: The current system relies entirely on PCA-transformed features (V1–V28), meaning the original transaction attributes (merchant, location, time-of-day patterns) are not directly interpretable. The model cannot leverage behavioral context such as spending habits or device fingerprints without access to the raw, pre-anonymized data.

12. Applications

The machine learning classification model developed in this project has several highly practical applications within the modern healthcare and medical technology ecosystem:

12.1 Real-Time Fraud Prevention System

The primary application of this system is to act as a real-time fraud scoring engine integrated into banking authorization pipelines. When a transaction is initiated, the model scores it within milliseconds, enabling banks to automatically flag or block high-probability fraud before processing, dramatically reducing financial losses.

12.2 Automated Transaction Triage

In high-volume transaction environments, manual fraud review is impossible at scale. This model can be integrated as an automated triage tool. Transactions flagged as fraudulent with high confidence are held for analyst review, while low-risk transactions flow through instantly, optimizing both security and customer experience.

12.3 Fraud Analytics Dashboard

Access to advanced fraud detection systems is often limited to large banks with significant IT budgets. By deploying this lightweight ML model as a cloud-based API or microservice, smaller financial institutions, credit unions, and fintech startups can access enterprise-grade fraud protection without building expensive in-house ML infrastructure.

12.4 Training Tool for Fraud Analysts

Because this system utilizes feature importance mapping (clearly defining which transaction characteristics like *V14 and V4*, *V4 and V12*, and *V17* drive fraud detection), it serves as an excellent training tool for fraud analysts and compliance teams. Junior investigators can use the system to study how different transaction patterns correlate with fraud, developing stronger intuition for anomaly detection.

13. Future Scope

While the current system achieves strong performance on transaction feature data, there are several promising avenues for future enhancement to make it more robust and commercially viable:

13.1 Deep Learning and Sequential Modeling

The current architecture uses PCA-transformed static features. A significant future enhancement would be integrating deep learning models such as LSTMs or Autoencoders to directly process raw transaction sequences. By modeling temporal spending behavior and merchant category patterns, a sequence-aware model could detect novel fraud schemes that evade pattern-based detection.

13.2 Cloud Deployment and Real-Time API

To transition this project into a production-ready banking tool, the next step involves cloud deployment. By hosting the model on platforms like AWS or Google Cloud and wrapping it in a secure RESTful API, financial institutions can connect their payment processing systems directly to the prediction engine for real-time fraud scoring on every transaction.

13.3 Multi-Domain Fraud Generalization

Currently, the model is tuned for credit card fraud detection. Future iterations will aim to build a generalized financial fraud framework. By incorporating behavioral data, geolocation, and merchant patterns, detection capabilities can be extended to wire transfer fraud, e-commerce fraud, and identity theft scenarios.

13.4 Advanced Explainable AI for Compliance

While the current model utilizes global feature importance plots, future versions will integrate advanced Explainable AI frameworks like SHAP (SHapley Additive exPlanations) or LIME. These tools will provide "local explanations"—meaning the AI will explicitly highlight exactly why it made a specific prediction for an *individual transaction*, detailing exactly how much each transaction feature contributed to the fraud risk score. This will significantly increase analyst trust and support regulatory compliance requirements.

14. Conclusion

This project successfully engineered and evaluated a machine learning classification system designed to detect credit card fraud with high precision and recall. By systematically addressing the challenge of extreme class imbalance through undersampling and SMOTE strategies, and evaluating Logistic Regression and Decision Tree classifiers, the system demonstrated strong detection performance across all tested conditions.

The Decision Tree model with undersampled data achieved the best F1 Score of 0.940, with Precision of 96.9% and Recall of 91.3%. Beyond pure predictive power, the project demonstrated that raw accuracy is a misleading metric for imbalanced datasets, and that balanced sampling is essential to build truly effective fraud detectors.

Ultimately, this project highlights the immense potential of Machine Learning not as a replacement for fraud analysts, but as a force-multiplying tool. By automating the screening of millions of daily transactions, reducing false positives, and providing interpretable fraud signals, ML-enhanced systems represent the future of financial security. AI-augmented fraud detection will be instrumental in reducing losses, protecting cardholders, and maintaining trust in digital payment ecosystems globally.

15. References

- Credit Card Fraud Detection Dataset, Kaggle. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

Scikit-learn Documentation and User Guide.

Chawla, N. V. et al. SMOTE: Synthetic Minority Over-sampling Technique. Journal of AI Research, 2002.

16. Appendix

A. Dataset Samples & Data Visualization

A. Data Loading and Exploratory Data Analysis

The dataset was loaded from creditcard.csv containing 284,807 transactions with 31 columns (Time, V1-V28, Amount, Class).

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
data = pd.read_csv("creditcard.csv")

print(data.shape)          # Output: (284807, 31)
print(data.info())

# Check class distribution
print(data["Class"].value_counts())
# Output:
# Class
# 0    284315   (Normal)
# 1      492   (Fraud)

# Visualize class distribution
sns.countplot(x=data["Class"])
plt.title("Class Distribution")
plt.show()
```

B. Data Preprocessing

StandardScaler was applied to the Amount column. The Time column was dropped. Duplicate rows were removed.

```
# Standardize Amount column
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
data["Amount"] = sc.fit_transform(pd.DataFrame(data["Amount"]))

# Drop Time column
data = data.drop(["Time"], axis=1)

# Remove duplicates
data = data.drop_duplicates()
print(data.shape)    # Output: (283726, 30)
```

```
#Verifyclass distribution after cleaning
print(data["Class"].value_counts())
# Class
#0      283253
#1       473
```

C. Model Training on Imbalanced Data

Initial models trained on the original imbalanced dataset. Accuracy is misleading here due to class imbalance.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score)

X = data.drop("Class", axis=1)
y = data["Class"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

classifiers = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier()
}

for name, clf in classifiers.items():
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    print(f"\n=== {name} ===")
    print(f"Accuracy : {accuracy_score(y_test, y_pred):.4f}")
    print(f"Precision: {precision_score(y_test, y_pred):.4f}")
    print(f"Recall    : {recall_score(y_test, y_pred):.4f}")
    print(f"F1 Score  : {f1_score(y_test, y_pred):.4f}")

# === Logistic Regression ===
# Accuracy : 0.9987 Precision: 0.6222
# Recall    : 0.5714 F1 Score : 0.5957
#
# === Decision Tree ===
# Accuracy : 0.9991 Precision: 0.6949
# Recall    : 0.8367 F1 Score : 0.7593
```

D. Undersampling and Model Training on Balanced Data

Random undersampling was applied to create a balanced dataset. Both models were retrained and showed significantly improved F1 Scores.

```
# Separate classes
normal = data[data["Class"] == 0]
fraud = data[data["Class"] == 1]

# Undersample normal class to match fraud count (473)
normal_sample = normal.sample(n=473, random_state=42)
balanced_data = pd.concat([normal_sample, fraud], ignore_index=True)

print(balanced_data["Class"].value_counts())
# Class
# 0      473
#1      473 (now perfectly balanced)

X2 = balanced_data.drop("Class", axis=1)
y2 = balanced_data["Class"]
X2_train, X2_test, y2_train, y2_test = train_test_split(
    X2, y2, test_size=0.2, random_state=42)
```

```

for name, clf in classifiers.items():
    clf.fit(X2_train, y2_train)
    y_pred = clf.predict(X2_test)
    print(f"\n=== {name} (Undersampled) ===")
    print(f"Accuracy : {accuracy_score(y2_test, y_pred):.4f}")
    print(f"Precision: {precision_score(y2_test, y_pred):.4f}")
    print(f"Recall    : {recall_score(y2_test, y_pred):.4f}")
    print(f"F1 Score : {f1_score(y2_test, y_pred):.4f}")

# === Logistic Regression (Undersampled) ===
# Accuracy : 0.9223 Precision: 0.9783
# Recall    : 0.8738 F1 Score : 0.9231
#
# === Decision Tree (Undersampled) ===
# Accuracy : 0.9378 Precision: 0.9691
# Recall    : 0.9126 F1 Score : 0.9400 <- Best Result

```

E. SMOTE Oversampling and Model Training

SMOTE was applied to generate synthetic fraud samples, creating a dataset with 284,315 instances per class.

```

from imblearn.over_sampling import SMOTE

X = data.drop("Class", axis=1)
y = data["Class"]

# Apply SMOTE
X_res, y_res = SMOTE(random_state=42).fit_resample(X, y)
print(pd.Series(y_res).value_counts())
# Class
# 0    284315
# 1    284315    (synthetic fraud samples added)

X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(
    X_res, y_res, test_size=0.2, random_state=42)

# Train Decision Tree on SMOTE data
dtc = DecisionTreeClassifier(random_state=42)
dtc.fit(X_train_s, y_train_s)
y_pred_s = dtc.predict(X_test_s)

print(f"Accuracy : {accuracy_score(y_test_s, y_pred_s):.4f}")
print(f"Precision: {precision_score(y_test_s, y_pred_s):.4f}")
print(f"Recall    : {recall_score(y_test_s, y_pred_s):.4f}")
print(f"F1 Score : {f1_score(y_test_s, y_pred_s):.4f}")

# Accuracy : 0.9275 Precision: 0.9684
# Recall    : 0.8932 F1 Score : 0.9293

```

F. Model Saving and Prediction

The best-performing Decision Tree model was saved using joblib for deployment and tested on a new transaction input.

```

import joblib

# Retrain best model on full SMOTE data
dtc_final = DecisionTreeClassifier(random_state=42)
dtc_final.fit(X_res, y_res)

# Save model
joblib.dump(dtc_final, "credit_card_fraud_model.pkl")

# Load model and predict on new transaction
model = joblib.load("credit_card_fraud_model.pkl")

# Sample transaction features (V1-V28 + Amount)

```

```
new_transaction = [[
    1.167, 0.502, -0.067, 2.262, 0.429, 0.089,
    0.241, 0.138, -0.989, 0.922, 0.745, -0.531,
    -2.105, 1.127, 0.003, 0.424, -0.454, -0.099,
    -0.817, -0.307, 0.019, -0.062, -0.104, -0.370,
    0.603, 0.109, -0.041, -0.011, 2.28
]]

prediction = model.predict(new_transaction)

if prediction[0] == 0:
    print("Result: Normal Transaction")
else:
    print("Result: FRAUDULENT Transaction - ALERT!")

# Output: Result: Normal Transaction
```